



Islington college
(इस्लिङ्टन कलेज)

Module Code & Module Title

CS6P05NI Project

Interim Report

25% Individual Coursework

Submission: Final Submission

AY 2025 2026

Academic Semester: Autumn Semester 2025

Credit: 30 Credit Year Long Module

Student Name: Resha Shrestha

London Met ID: 23048661

College ID: 230121

Assignment Due Date: Tuesday, December 30, 2025

Assignment Submission Date: Tuesday, December 30, 2025

Internal Supervisor: Mr. Roshan Shrestha

External Supervisor: Ms. Kriti Dangol

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

1. INTRODUCTION

1.1 Introduction to the Topic

The education landscape has undergone significant transformation with digital learning becoming crucial for academic success. Despite the availability of numerous digital tools, students continue to face fundamental study challenges. Modern learners encounter fragmented workflows, relying on separate applications for reading PDFs, taking notes, creating flashcards, and practicing with questions. This fragmentation disrupts cognitive flow and leads to inefficiency.

EduAI Assistant addresses these challenges by providing an integrated AI-powered educational platform that combines document processing, intelligent summarization, interactive question-answering, and automated assessment generation into a unified learning ecosystem. This project leverages transformer-based models and natural language processing to create a comprehensive solution that enhances both learning efficiency and educational outcomes.

1.2 Current Scenario and Issues/Challenges

1.2.1 Global Issues

Globally, the educational technology landscape faces several critical challenges:

- **Fragmented Learning Tools:** Students typically use 5-7 different applications for studying, leading to context switching and reduced productivity
- **Time-Intensive Manual Tasks:** Creating summaries, flashcards, and practice questions consumes significant study time that could be spent on actual learning
- **Passive Learning Methods:** Traditional study approaches often rely on passive reading rather than active engagement with materials
- **Limited Personalization:** Most educational tools provide one-size-fits-all solutions without adapting to individual learning needs
- **Assessment Quality:** Manual question generation often lacks the cognitive depth needed for effective learning assessment

1.2.2 Local Issues (Nepal Context)

In Nepal's educational context, additional challenges compound these global issues:

- **Limited Access to Premium Tools:** High subscription costs for international educational platforms make them inaccessible for many Nepali students

- **Resource Constraints:** Students often lack access to quality study materials and comprehensive learning resources
- **Digital Literacy Gap:** Many students struggle with effectively utilizing available educational technology
- **Exam-Oriented Learning:** The focus on rote memorization rather than conceptual understanding limits the effectiveness of traditional study methods

1.3 Problem Statement

Students face significant challenges in their learning journey due to fragmented study workflows, time-consuming manual task creation, and limited access to personalized learning tools. The current educational technology landscape lacks integrated solutions that combine intelligent document processing, active learning methodologies, and adaptive assessment generation. This results in inefficient study sessions, reduced engagement, and suboptimal learning outcomes. There is a critical need for an affordable, comprehensive platform that leverages artificial intelligence to streamline the entire study process while promoting evidence-based active learning techniques.

1.4 Project as a Solution

EduAI Assistant addresses these challenges through a unified, AI-driven approach that transforms the entire study process:

Unified Learning Ecosystem:

The platform provides a single interface for the complete study workflow - uploading documents, generating summaries, asking questions, creating flashcards, and taking practice tests. This integration eliminates context switching and creates a seamless learning experience.

Intelligent Document Processing:

Using FLAN-T5-base transformer models, the system generates coherent, contextually rich summaries that preserve educational significance and logical flow, going beyond simple sentence extraction.

Interactive Question-Answering:

The Retrieval-Augmented Generation (RAG) pipeline combines vector search with DistilBERT-squad to provide accurate, context-aware answers to student queries.

Automated Study Aid Generation:

Named Entity Recognition and natural language generation automatically create flashcards and practice questions, saving hours of manual work.

Adaptive Assessment:

The system generates questions at multiple difficulty levels aligned with Bloom's taxonomy, ensuring effective cognitive challenge and learning assessment.

1.5 Targeted End Users

The platform targets:

- University and College Students: Primary users seeking efficient study methods and comprehensive learning support
- Self-Learners: Individuals pursuing independent learning who need structured study assistance
- Competitive Exam Aspirants: Students preparing for standardized tests requiring efficient practice and assessment tools
- Educators and Teachers: Secondary users who can utilize the platform to generate teaching materials and assessments

Future Scope and Probable Clients:

Future expansion opportunities include partnerships with educational institutions, licensing to corporate training departments, integration with Learning Management Systems (LMS), and development of mobile applications for broader accessibility.

1.6 Aims and Objectives

Project Aim:

To develop an integrated AI-powered educational platform that enhances student learning efficiency through intelligent document processing, interactive question-answering, and automated assessment generation, ultimately improving educational outcomes through evidence-based active learning methodologies.

Project Objectives:

1. Implement core AI features using transformer-based models
2. Develop MVP summarization feature with PDF upload and intelligent text processing
3. Create interactive RAG-based question-answering system with semantic search
4. Generate automated flashcards using Named Entity Recognition and NLP
5. Implement multi-level MCQ generation system aligned with Bloom's taxonomy
6. Develop responsive React.js frontend with comprehensive user interface
7. Conduct user testing and validation with target audience
8. Document technical architecture and deployment procedures

2. BACKGROUND

2.1 Background/Literature Review

2.2 Project Overview and Justification

EduAI Assistant is justified by the growing demand for integrated, AI-powered educational tools that combine efficiency with pedagogical effectiveness. The platform addresses documented challenges in student learning workflows while leveraging proven AI architectures and evidence-based learning methodologies. Unlike existing solutions that focus on single aspects of studying, this project creates a comprehensive ecosystem that supports the entire learning journey.

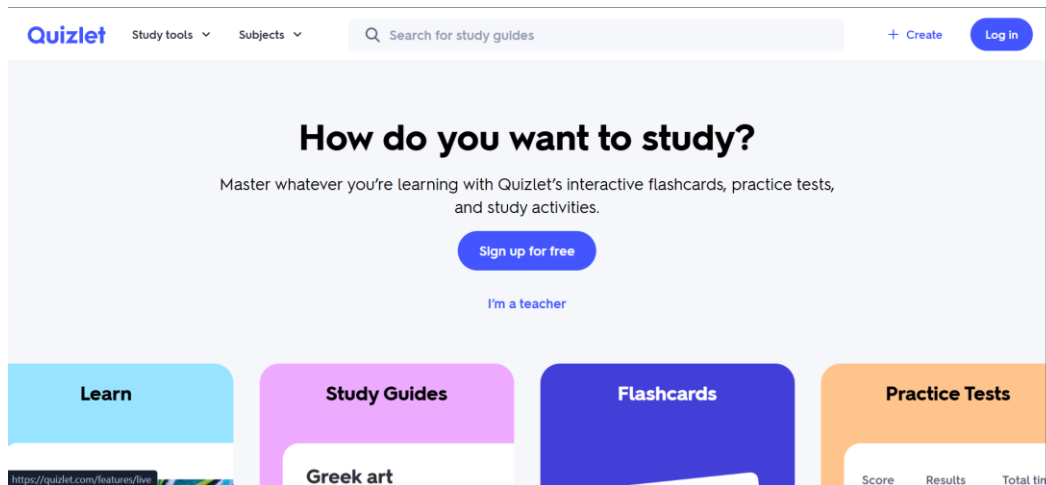
2.3 Deliverables

- Fully functional web application with AI-powered features
- Backend API implementing document processing, summarization, Q&A, and assessment generation
- Responsive React.js frontend with comprehensive user interface
- Vector database for semantic search and document retrieval
- Automated flashcard and MCQ generation systems
- Technical documentation including API specifications and deployment guides
- User testing results and validation report
- Complete project dissertation with methodology and results analysis

2.4 Similar Projects

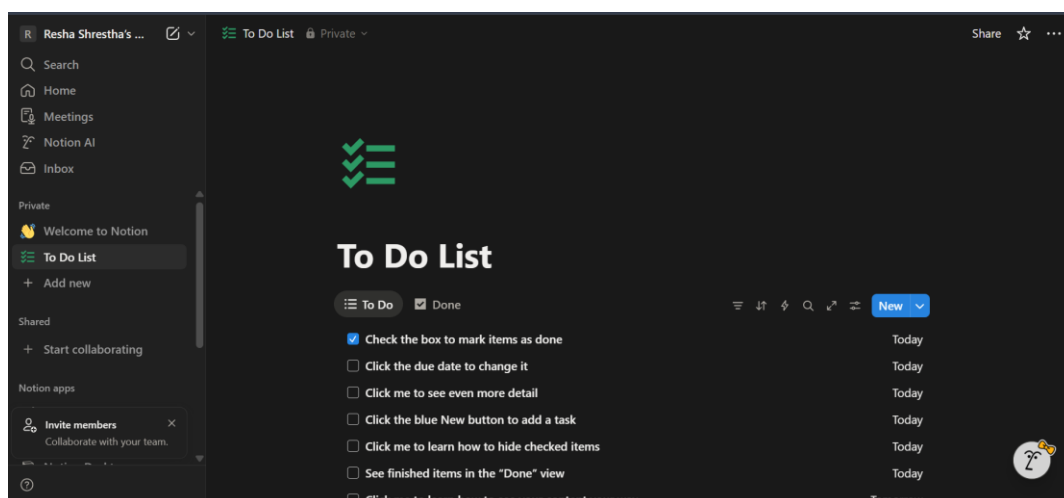
Project 1: Quizlet

Quizlet is a popular flashcard-based learning platform that allows users to create and study digital flashcards. It offers features like Learn mode, Test mode, and Match games. However, Quizlet primarily focuses on user-created content and lacks automated document processing and intelligent summarization capabilities. Its AI features are limited compared to the comprehensive NLP integration planned for EduAI Assistant.



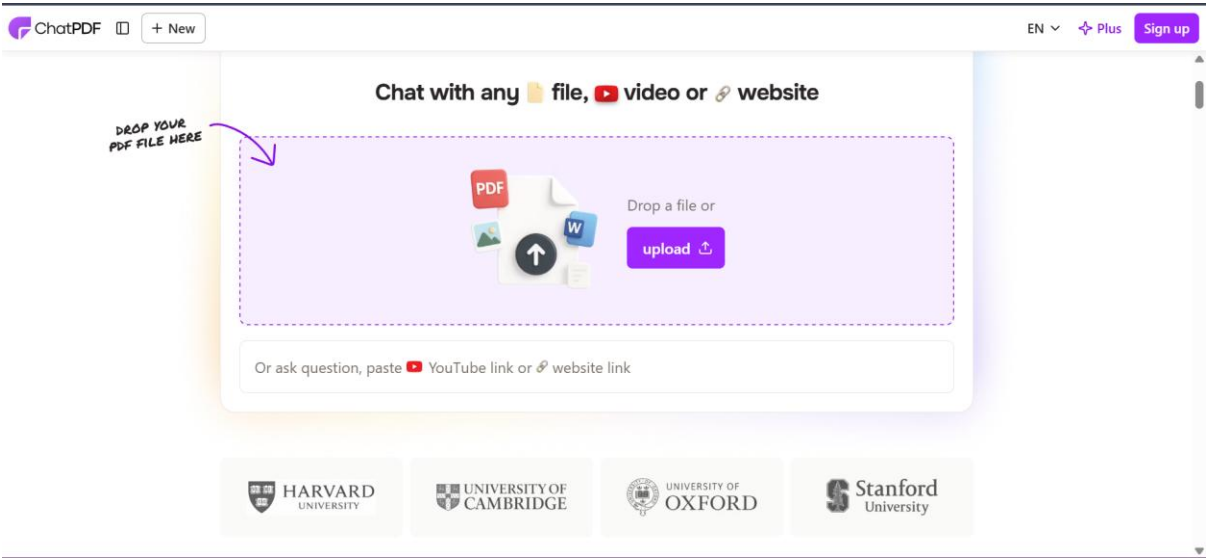
Project 2: Notion AI

Notion AI integrates AI writing assistance into Notion's workspace platform, offering features like text summarization and content generation. While powerful for general productivity, Notion AI is not specifically designed for educational workflows and lacks specialized features like RAG-based Q&A, adaptive assessment generation, and evidence-based learning methodologies that EduAI Assistant provides.



Project 3: ChatPDF

ChatPDF enables users to chat with PDF documents using AI. It excels at question-answering but offers limited features for active learning, such as flashcard generation, practice quiz creation, or comprehensive study aid automation. EduAI Assistant provides a more complete learning ecosystem beyond simple document interaction.



Comparative Analysis:

Comprehensive feature comparison between EduAI Assistant and existing educational platforms

Feature	Quizlet	Notion AI	ChatPDF	EduAI Assistant
Document Upload	X	✓	✓ (PDF only)	✓ (PDF)
AI Summarization	X	Basic	X	✓ (FLAN-T5)
Q&A System	X	X	✓ (Basic)	✓ (RAG)
Flashcard Generation	Manual Only	X	X	✓ (Auto)
MCQ Generation	Manual Only	X	X	✓ (3 Levels)
Adaptive Difficulty	X	X	X	✓ (Bloom's Taxonomy)
Source Citations	X	X	✓	✓
Progress Analytics	✓ (Basic)	X	X	✓ (Advanced)
Unified Platform	Partial	X	X	✓

Feature	Quizlet	Notion AI	ChatPDF	EduAI Assistant
AI Technology	Basic NLP	GPT-based	GPT-based	FLAN-T5 + RAG
Primary Focus	Flashcards	Productivity	PDF Chat	All in one

EduAI Assistant distinguishes itself by combining features from all three platforms while adding unique capabilities. Unlike Quizlet's manual content creation, this platform automates flashcard generation. Unlike Notion AI's general-purpose approach, the platform focuses specifically on educational workflows with evidence-based learning principles. Unlike ChatPDF's limited interaction model, this platform provides comprehensive study tools including summarization, multiple assessment types, and adaptive difficulty levels. The integration of transformer models specifically fine-tuned for educational tasks sets EduAI Assistant apart as a specialized, comprehensive learning platform.

2.5 Methodology

METHODOLOGY COMPARISON

2.2 Software Development Methodology

2.2.1 Considered Methodologies

Several software development methodologies were evaluated for their suitability to the EduAI Assistant project. The key methodologies considered were Waterfall, Iterative, and Scrum (Agile). Each methodology was assessed based on its ability to accommodate the project's unique requirements, including AI model integration, iterative testing, supervisor collaboration, and academic timeline constraints.

Waterfall Methodology:

The Waterfall methodology follows a linear, sequential approach where each phase (Requirements, Design, Implementation, Testing, Deployment, Maintenance) must be completed before the next begins. This traditional approach provides clear structure with well-defined deliverables, comprehensive documentation at each stage, and straightforward project planning. However, Waterfall's rigid structure makes it difficult to accommodate requirement changes discovered during development, delays feedback until late in the project lifecycle, and struggles with the experimental nature of AI model integration where multiple iterations and adjustments are often necessary.

Iterative Methodology:

The Iterative methodology cycles repeatedly through planning, implementation, and evaluation phases, with each iteration building incrementally upon previous work. This approach allows for gradual system refinement, reduces project risk through incremental progress, and enables learning from earlier iterations to inform later development. However, without the structured framework of sprints and defined ceremonies, iterative development can lead to scope creep if not carefully managed, may lack clear timelines for deliverables, and can become disorganized without proper planning discipline.

Scrum Methodology (Agile):

Scrum is an Agile framework that organizes development into fixed-length sprints (typically 2-4 weeks), with defined ceremonies including sprint planning, daily standups, sprint reviews, and retrospectives. Scrum emphasizes iterative development, continuous feedback, cross-functional collaboration, and adaptive planning. The methodology balances structure with flexibility, making it particularly suitable for projects involving emerging technologies, uncertain requirements, or need for regular stakeholder engagement.

2.2.2 Selected Methodology

For this project, **Scrum** was chosen as the preferred methodology for software development. The justification as to why Scrum was chosen for the project's methodology is discussed below through case study scenarios that illustrate how Scrum's features address specific project requirements.

Case Study Scenario 1: Fostering Collaboration with Supervisors and Domain Experts

Case Study Scenario	Fostering collaboration between developers and educational technology experts
Features	Scrum encourages stakeholder involvement through sprint reviews and regular demonstrations.
Justification	Regular sprint reviews with internal and external supervisors (Mr. Roshan Shrestha and Mr. Kriti Dangol) ensure that features like AI summarization, question-answering, and assessment generation align with educational best practices and pedagogical principles. This continuous feedback loop

	validates that the system meets actual student needs and academic requirements.
--	---

Table 2: Case Study Scenario - Supervisor Collaboration

Case Study Scenario 2: Developing an Educational System that Adapts to Evolving Requirements

Case Study Scenario	Developing an educational AI system that adapts to user's evolving requirements and emerging AI capabilities.
Features	Scrum's iterative approach is flexible to accommodate changing requirements that might arise during development based on user feedback, technological advances, or new insights.
Justification	During AI model experimentation, initial testing may reveal that FLAN-T5 parameters need adjustment, or that a different chunking strategy improves summarization quality. Scrum allows for constant iteration and updates based on student user testing insights, ensuring the platform remains relevant and effective. For example, if user testing reveals that flashcard difficulty levels need recalibration, this can be incorporated into the next sprint without derailing the entire project.

Table 3: Case Study Scenario - Evolving Requirements

Case Study Scenario 3: Ensuring Timely Delivery of Features

Case Study Scenario	Ensuring timely delivery of features within academic semester constraints and interim report deadlines.
Features	Scrum delivers features in small, manageable time frames known as sprints (typically 2 weeks).
Justification	Scrum's sprint-based delivery ensures that essential features are made functional in the early development cycle, enabling

	demonstration of working software for reports and supervisor meetings.
--	--

Table 4: Case Study Scenario - Timely Delivery

Case Study Scenario 4: Managing the Complexity of Features in the System

Case Study Scenario	Managing the complexity of features involving AI model integration, RAG pipeline, and multiple interconnected components.
Features	Scrum breaks down complex tasks into manageable sprint goals and user stories with clear acceptance criteria.
Justification	Complex features like the RAG pipeline (involving document chunking, embedding generation, FAISS vector database setup, and answer generation), automated flashcard generation (requiring NER, keyword extraction, and content generation), and multi-level MCQ creation can be broken down into smaller, focused deliverables. Each sprint can tackle specific components, allowing for concentrated development, thorough testing, and incremental integration. This decomposition reduces cognitive load, enables parallel work on different components, and makes complex AI integration more manageable.

Table 5: Case Study Scenario - Managing Complexity

Case Study Scenario 5: Facilitating AI Model Experimentation and Optimization

Case Study Scenario	Facilitating experimentation with different AI models, hyperparameters, and architectural approaches to optimize performance.
Features	Scrum's retrospective ceremonies and iterative cycles support continuous learning and experimentation.
Justification	AI development inherently involves experimentation testing different model variants (FLAN-T5-small vs. base vs. large),

	adjusting hyperparameters (temperature, max_length, beam search), and optimizing prompts. Sprint retrospectives provide structured opportunities to review what worked, what didn't, and what to try next
--	---

Table 6: Case Study Scenario - AI Experimentation

Conclusion

Scrum's combination of structured sprints, regular stakeholder engagement, iterative development, and flexible adaptation makes it the optimal methodology for the EduAI Assistant project. The framework's emphasis on delivering working software incrementally aligns perfectly with academic timelines requiring demonstrable progress at interim stages. The collaborative nature of Scrum facilitates essential feedback from supervisors and domain experts, ensuring the system meets both technical and pedagogical standards. Most importantly, Scrum's iterative approach accommodates the experimental nature of AI development, where optimization and refinement are ongoing processes rather than one-time activities.

The case study scenarios demonstrate how Scrum addresses specific challenges inherent to educational AI development: fostering expert collaboration, adapting to evolving requirements, ensuring timely delivery, managing complexity, and facilitating experimentation. These scenarios validate Scrum as not just a suitable choice, but the ideal methodology for successfully delivering the EduAI Assistant platform within the constraints and requirements of an academic capstone project.

2.6 Technology Used

2.6.1 Programming Languages

Python 3.10+

Python serves as the primary backend language for EduAI Assistant due to its exceptional ecosystem for artificial intelligence and machine learning development. The language provides access to industry-leading libraries such as transformers, PyTorch, sentence-transformers, and spaCy, which offer production-ready implementations of state-of-the-art models. Python's clear syntax and rapid development capabilities align perfectly with the Agile Scrum methodology, enabling quick iterations and prototyping. Modern Python features including type hints, async/await syntax, and dataclasses enhance code maintainability and

performance. The language's extensive documentation and active community support facilitate problem-solving during development.

JavaScript/TypeScript

JavaScript powers the frontend through React.js, enabling the creation of dynamic, responsive user interfaces with excellent performance. The language's event-driven nature suits the interactive requirements of the platform, handling user inputs, real-time updates, and asynchronous operations seamlessly. TypeScript integration provides static type checking, reducing runtime errors and improving code quality in complex components. The Node.js ecosystem offers robust tooling for development, testing, building, and deployment, including package managers (npm/yarn), build tools (Webpack, Vite), and testing frameworks (Jest, React Testing Library).

2.6.2 Frontend Framework

React.js

React's component-based architecture enables modular, reusable UI development that scales effectively as the application grows. The virtual DOM ensures efficient rendering, critical for dynamic content updates when displaying summaries, questions, and flashcards. React Hooks (useState, useEffect, useContext, useReducer) provide elegant state management without the complexity of class components, simplifying code and reducing boilerplate. The extensive ecosystem includes React Router for navigation, Axios for API communication, and UI libraries (Material-UI, Tailwind CSS) for rapid interface development. React's developer tools and hot reloading capabilities accelerate development and debugging. The framework's popularity ensures abundant learning resources, community support, and availability of skilled developers for potential future expansion.

2.6.3 Backend Framework

FastAPI

FastAPI offers modern async capabilities essential for handling concurrent requests efficiently, automatic API documentation through OpenAPI/Swagger, and excellent performance comparable to Node.js frameworks. The framework integrates seamlessly with Python ML libraries, enabling smooth communication between web endpoints and AI models. The RESTful API design ensures clear separation between frontend and backend, facilitating independent development, testing, and potential future service decomposition.

2.6.4 Database

MySQL

MySQL serves as the primary relational database management system for EduAI Assistant, providing robust data storage with proven reliability and performance. As one of the most widely deployed open-source databases, MySQL offers comprehensive SQL support with ACID compliance ensuring data integrity critical for storing user information, documents, and assessment results. The database's mature ecosystem includes extensive documentation, active community support, and numerous management tools (MySQL Workbench, phpMyAdmin, HeidiSQL) facilitating development and administration.

MySQL's InnoDB storage engine provides transactional support with row-level locking, enabling concurrent access without performance degradation. Foreign key constraints maintain referential integrity across related tables (users, documents, summaries, flashcards, questions), preventing orphaned records and ensuring data consistency. The database's indexing capabilities (B-tree indexes, full-text search indexes) optimize query performance for common operations like user lookups, document searches, and quiz result retrieval.

For development, MySQL can run locally using XAMPP, WAMP, or direct installation, providing a familiar environment for testing and debugging. Production deployment options include managed services (AWS RDS for MySQL, Google Cloud SQL, Azure Database for MySQL) offering automated backups, high availability, and scaling capabilities. MySQL's replication features support read replicas for improved query performance and disaster recovery scenarios. The database's UTF-8 character set support (utf8mb4) ensures proper handling of international characters and emojis in user content.

FAISS Vector Database

FAISS (Facebook AI Similarity Search) serves as the specialized vector database for semantic search, providing fast nearest-neighbor search essential for the RAG pipeline. The library implements optimized algorithms (IVF, HNSW) enabling efficient searching through millions of document embeddings with sub-second query times. FAISS supports multiple index types allowing optimization for specific use cases – flat indexes for exact search, IVF for balanced speed/accuracy, and HNSW for large-scale deployments. Its integration with sentence-transformers is seamless and battle-tested. FAISS can operate entirely in memory for maximum speed or persist to disk for larger datasets. The library's active development by Meta AI ensures continued optimization and feature additions.

2.6.5 AI/ML Libraries

Hugging Face Transformers

The transformers library provides unified access to thousands of pre-trained models from the Hugging Face Model Hub, including FLAN-T5 and DistilBERT. The library offers standardized APIs for model loading, tokenization, and inference, simplifying integration regardless of underlying model architecture. Built-in pipelines provide high-level interfaces for common tasks (summarization, question-answering) reducing implementation complexity. The library supports multiple frameworks (PyTorch, TensorFlow) providing flexibility in deployment. Regular updates ensure access to latest model architectures and optimizations. Extensive documentation and active community support facilitate problem-solving during development.

Sentence-Transformers

Sentence-transformers enables efficient semantic similarity search through optimized sentence embeddings. The library provides models specifically fine-tuned for semantic similarity tasks, offering superior performance compared to general-purpose embeddings. The all-MiniLM-L6-v2 model selected for this project balances quality and speed, generating 384-dimensional embeddings suitable for real-time applications. The library supports custom fine-tuning on domain-specific data for potential future improvements.

spaCy

spaCy provides industrial-strength Natural Language Processing for named entity recognition, essential for identifying key concepts in documents for flashcard and question generation. The library offers pre-trained models for entity recognition, part-of-speech tagging, and dependency parsing with state-of-the-art accuracy. spaCy's efficient implementation enables fast processing of large documents, crucial for maintaining responsive user experience. The pipeline architecture allows custom components for specialized processing. The library's linguistic annotations (noun chunks, named entities, sentence boundaries) inform intelligent content extraction. spaCy's active development and comprehensive documentation ensure long-term viability.

LangChain

LangChain orchestrates the RAG pipeline, managing document processing, retrieval, and generation workflows. The framework provides abstractions for common patterns (document loaders, text splitters, retrievers) reducing boilerplate code. LangChain's modular design enables easy experimentation with different components swapping vector stores, embeddings, or language models without architectural changes. The framework includes utilities for prompt engineering, output parsing, and chain composition, facilitating complex multi-step workflows. Memory management features support conversational interactions. LangChain's rapid development and active community contribute new integrations and best practices regularly.

2.6.6 Development Tools

Git & GitHub

Version control through Git enables systematic tracking of code changes, facilitating rollback if issues arise and providing complete development history. GitHub hosting enables backup, collaboration features (if team expands), and integrated issue tracking. Branch strategy (main/develop/feature branches) supports organized development aligned with Agile sprints. Commit messages document development decisions and rationale.

Visual Studio Code

VS Code serves as the integrated development environment, providing Python and JavaScript language support through extensions, intelligent code completion, integrated debugging, and Git integration. The editor's extensions for React, Python, and database management enhance productivity. Built-in terminal enables seamless command execution. Workspace settings ensure consistent development environment.

Google Colab

Google Colab (Colaboratory) serves as the primary platform for AI model experimentation, development, and testing, offering significant advantages for machine learning workflows. Colab provides free access to GPU and TPU hardware accelerators, essential for training and fine-tuning transformer models that would be prohibitively slow on CPU-only machines. The platform's pre-installed libraries (TensorFlow, PyTorch, transformers, scikit-learn) reduce setup time and dependency management complexity. Colab notebooks support interactive development with immediate code execution and inline visualization of results, facilitating rapid prototyping and iteration. Markdown cells enable comprehensive documentation within notebooks, explaining model architectures, hyperparameter choices, and experimental results alongside executable code.

The interactive nature of notebooks allows immediate visualization of results through matplotlib and seaborn plots, comparing model outputs side-by-side, and documenting findings with explanatory text. Colab notebooks can be easily exported to Python scripts for production integration, ensuring that experimental code transitions smoothly to the deployed application.

3. DEVELOPMENT TO DATE

3.1 Requirement Gathering

Requirements were gathered through multiple channels:

- Student Survey: Conducted with students to identify pain points in current study workflows
- Literature Review: Analysis of existing educational technology solutions and their limitations
- Supervisor Consultations: Regular meetings with internal and external supervisors for requirement validation
- Competitor Analysis: Evaluation of similar platforms to identify feature gaps

3.2 Software Requirement Specification

Functional Requirements:

- System shall accept PDF document uploads up to 50MB
- System shall generate summaries within 30 seconds for typical documents
- System shall provide accurate answers to natural language questions
- System shall generate minimum 10 flashcards per document
- System shall create MCQs at three difficulty levels
- System shall provide progress tracking and study analytics

Non-Functional Requirements:

- Performance: Less system response time for most operations
- Scalability: Support for concurrent users with cloud deployment
- Usability: Intuitive interface requiring minimal learning curve
- Security: Secure document storage and user authentication
- Reliability: 99% uptime with error handling and recovery mechanisms

3.3 System Design Diagrams

3.3.1 Data Flow Diagrams

Level 0 DFD (Context Diagram):

Level 1 DFD:

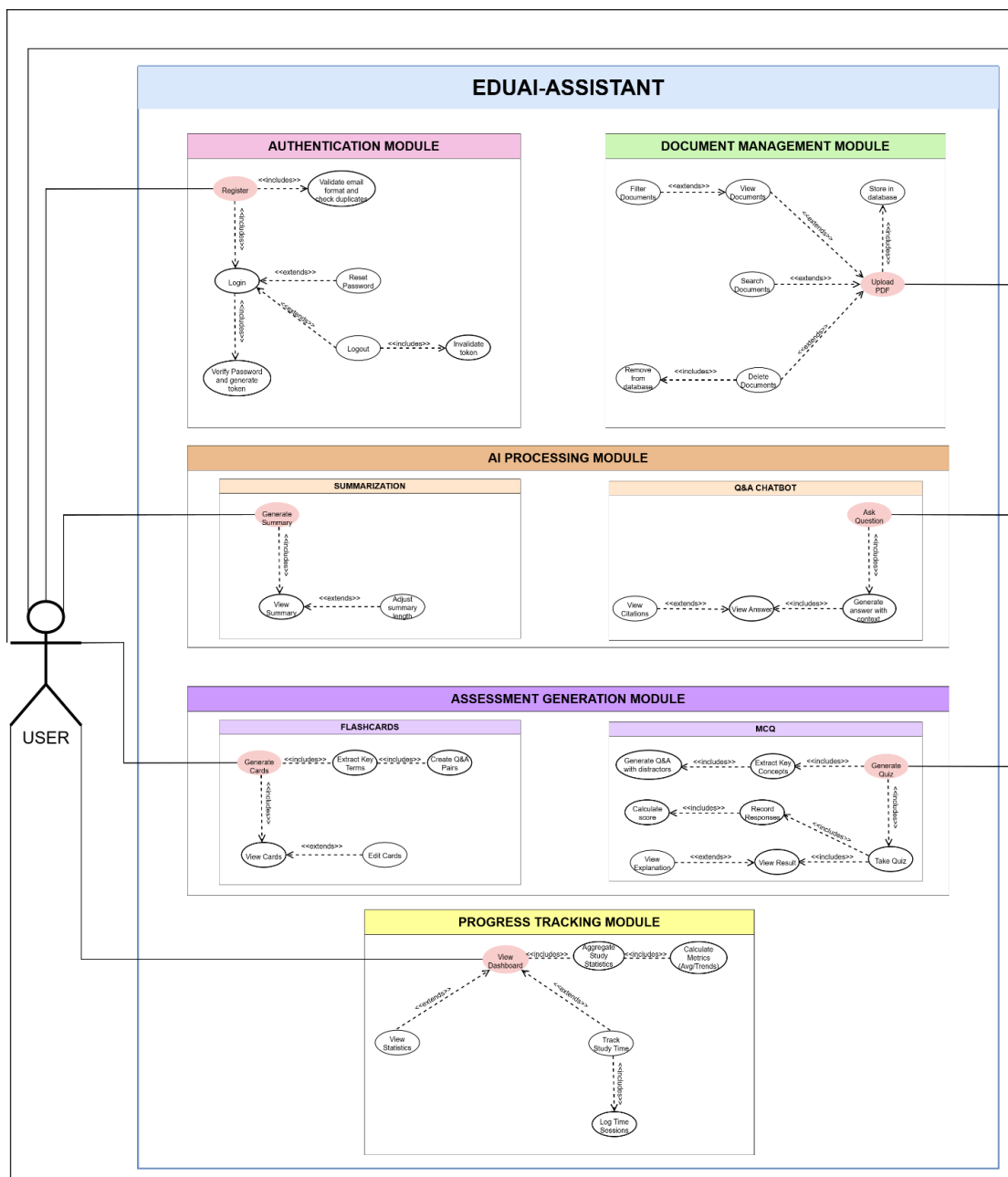
Level 2 DFD:

3.3.2 Use Case Diagrams

Primary actors: User, Admin

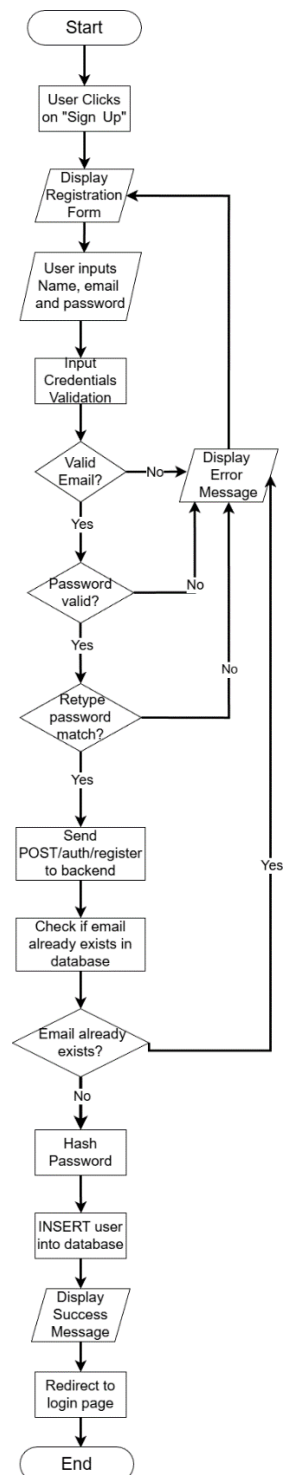
Key use cases include:

- Upload Document
- Generate Summary
- Ask Question
- Create Flashcards
- Generate Practice Quiz
- Review Progress
- Manage User Accounts (Admin)

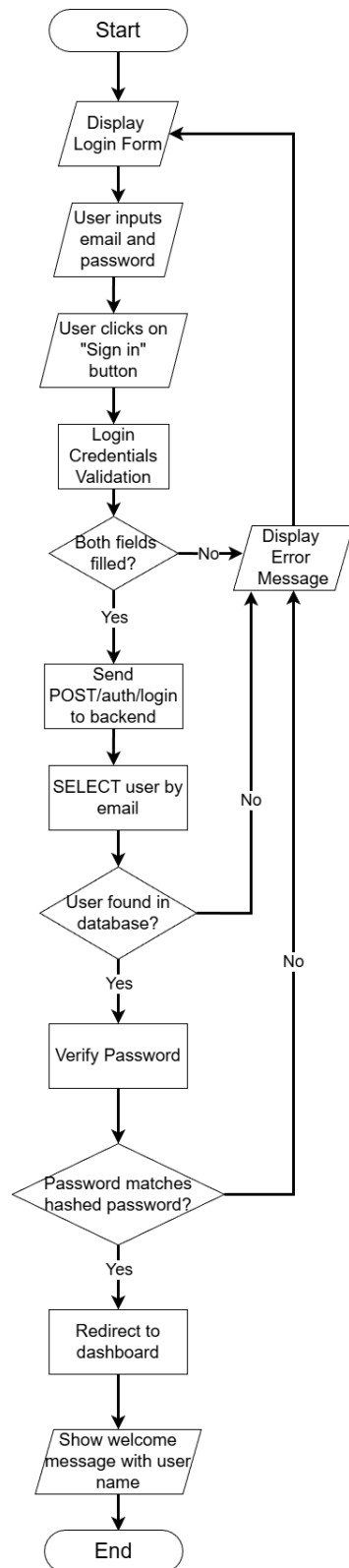


3.3.3 Flowcharts

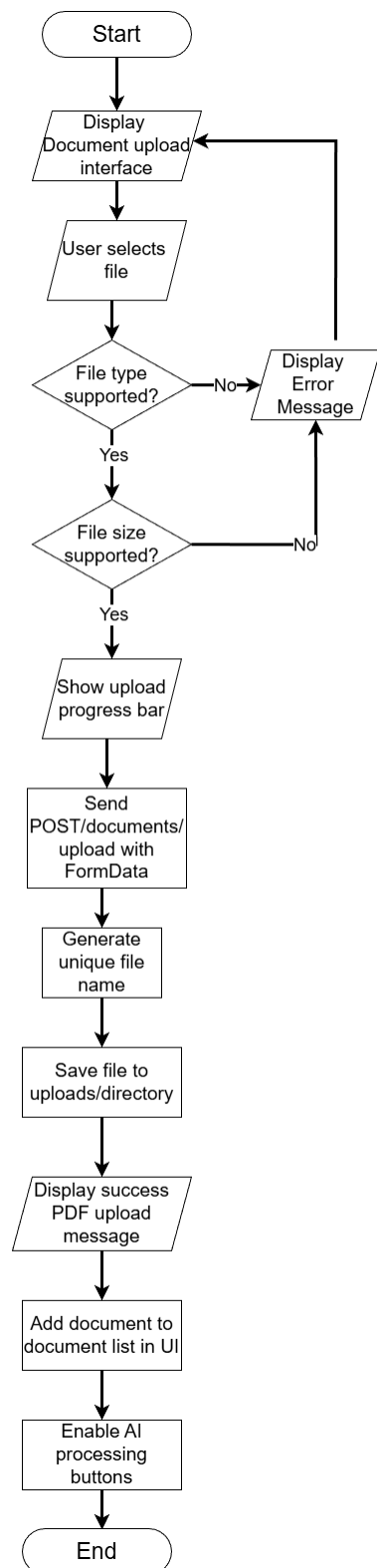
Registration Flowchart



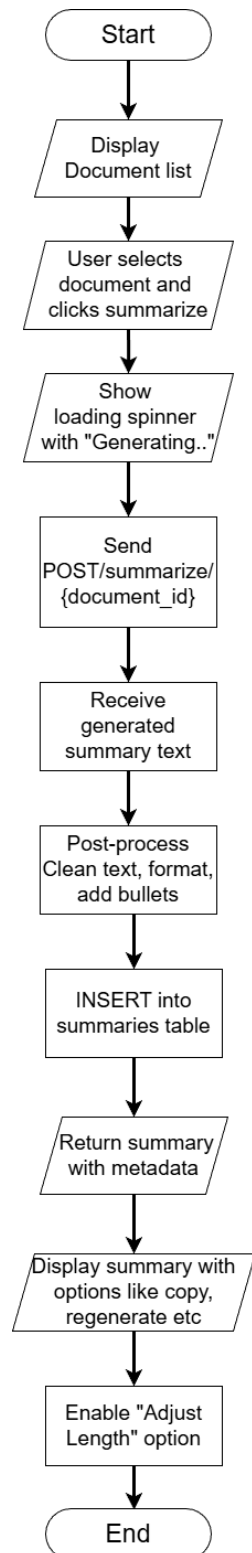
Login Flowchart



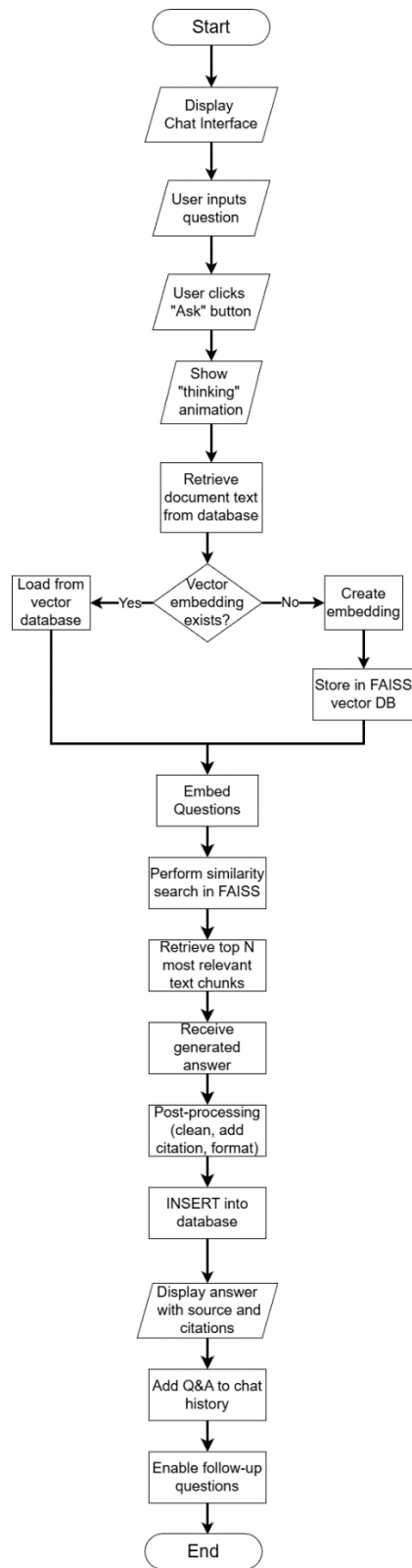
PDF Upload & Processing Flowchart



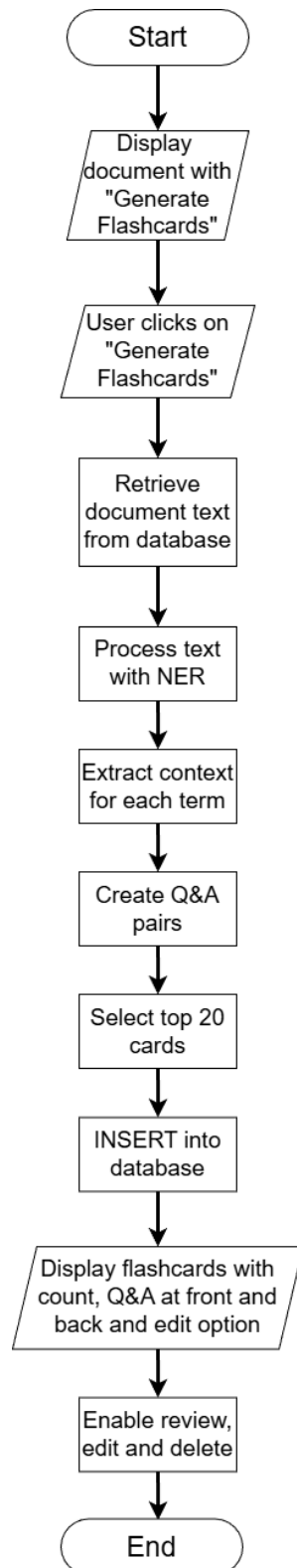
Document Summarization Flowchart



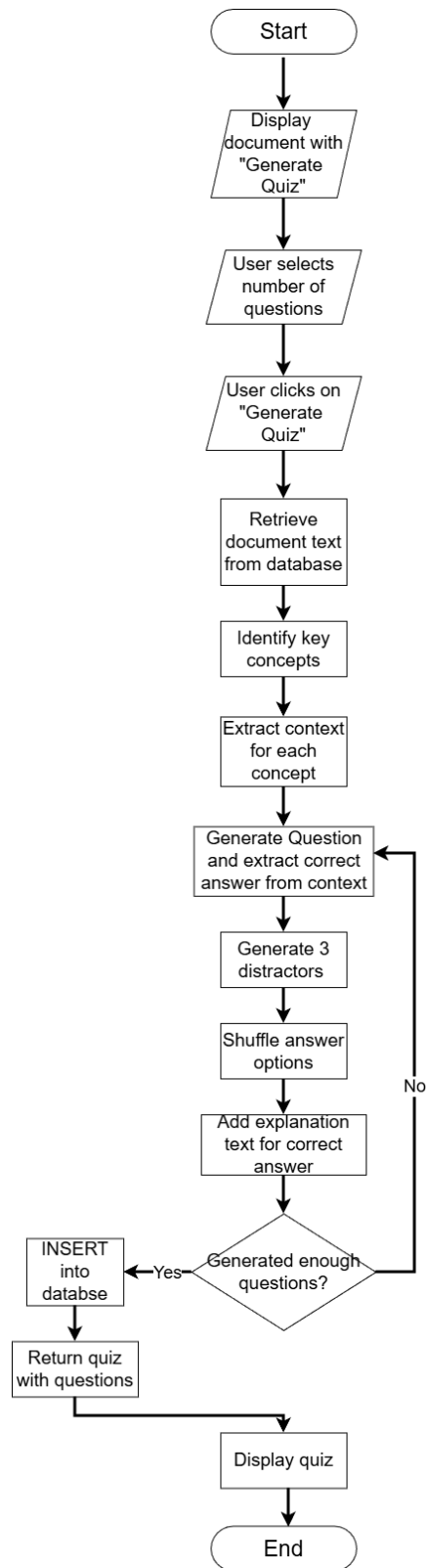
Q&A Chatbot Flowchart



Flashcard Creation Flowchart

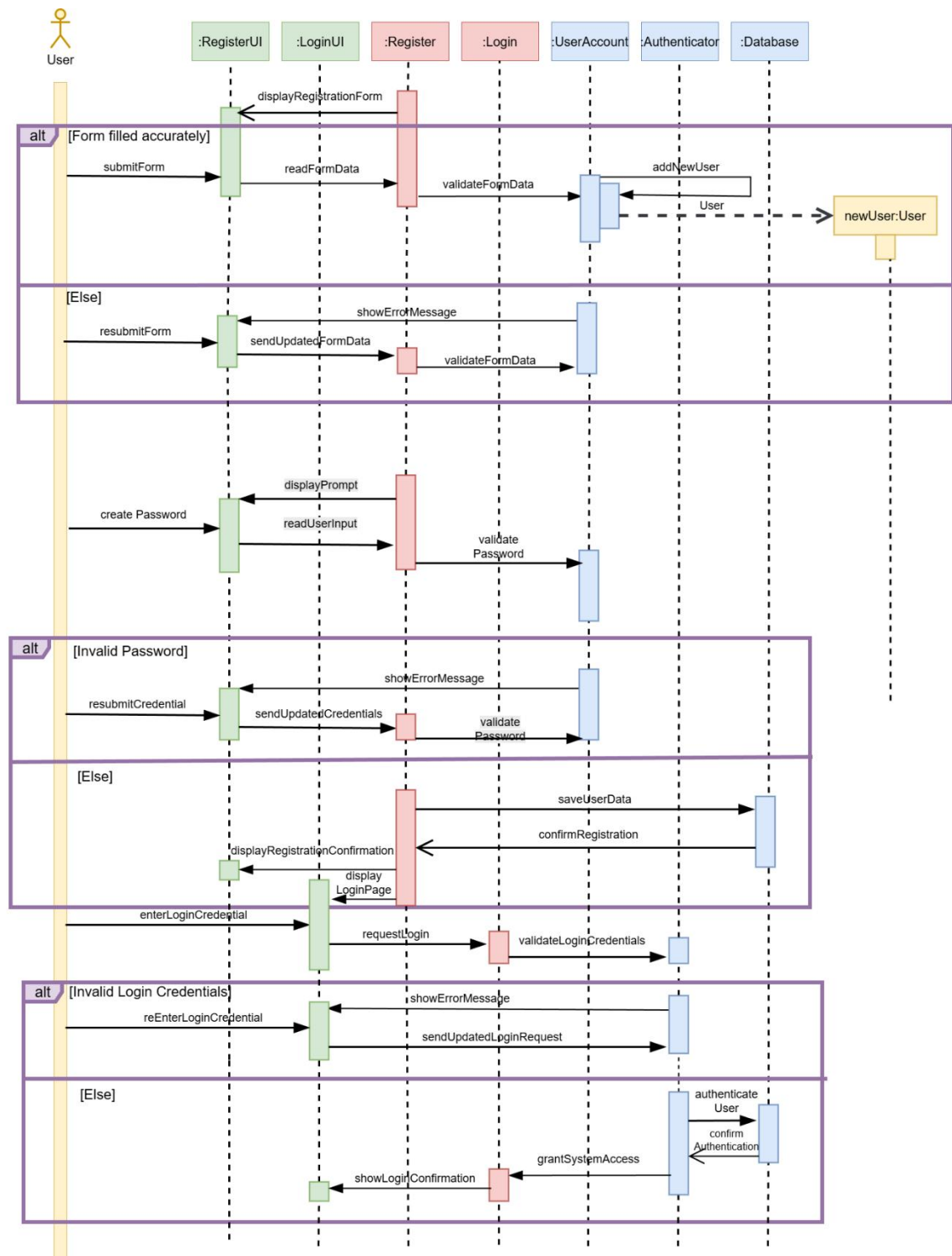


MCQ Generation Flowchart

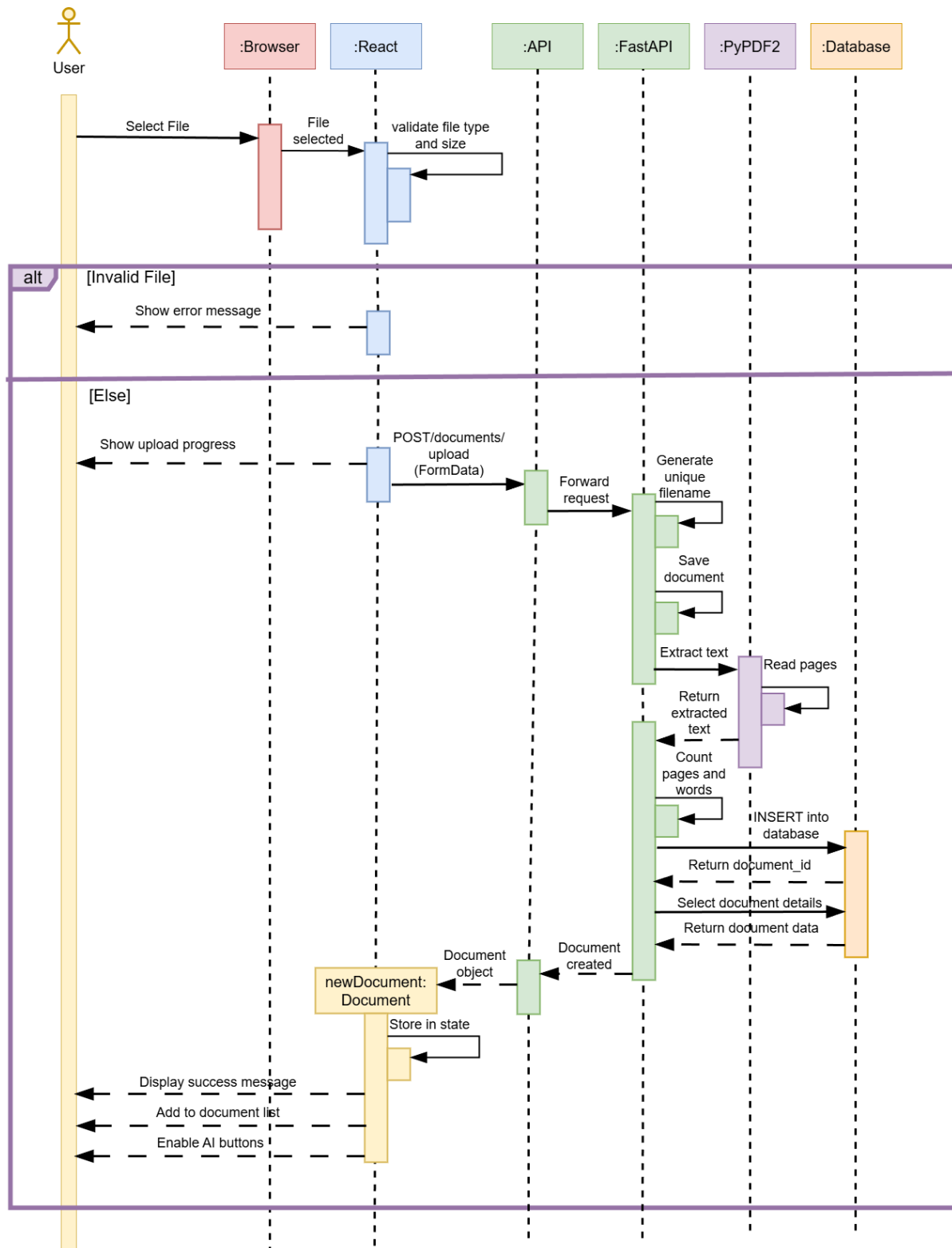


3.3.4 Sequence Diagrams

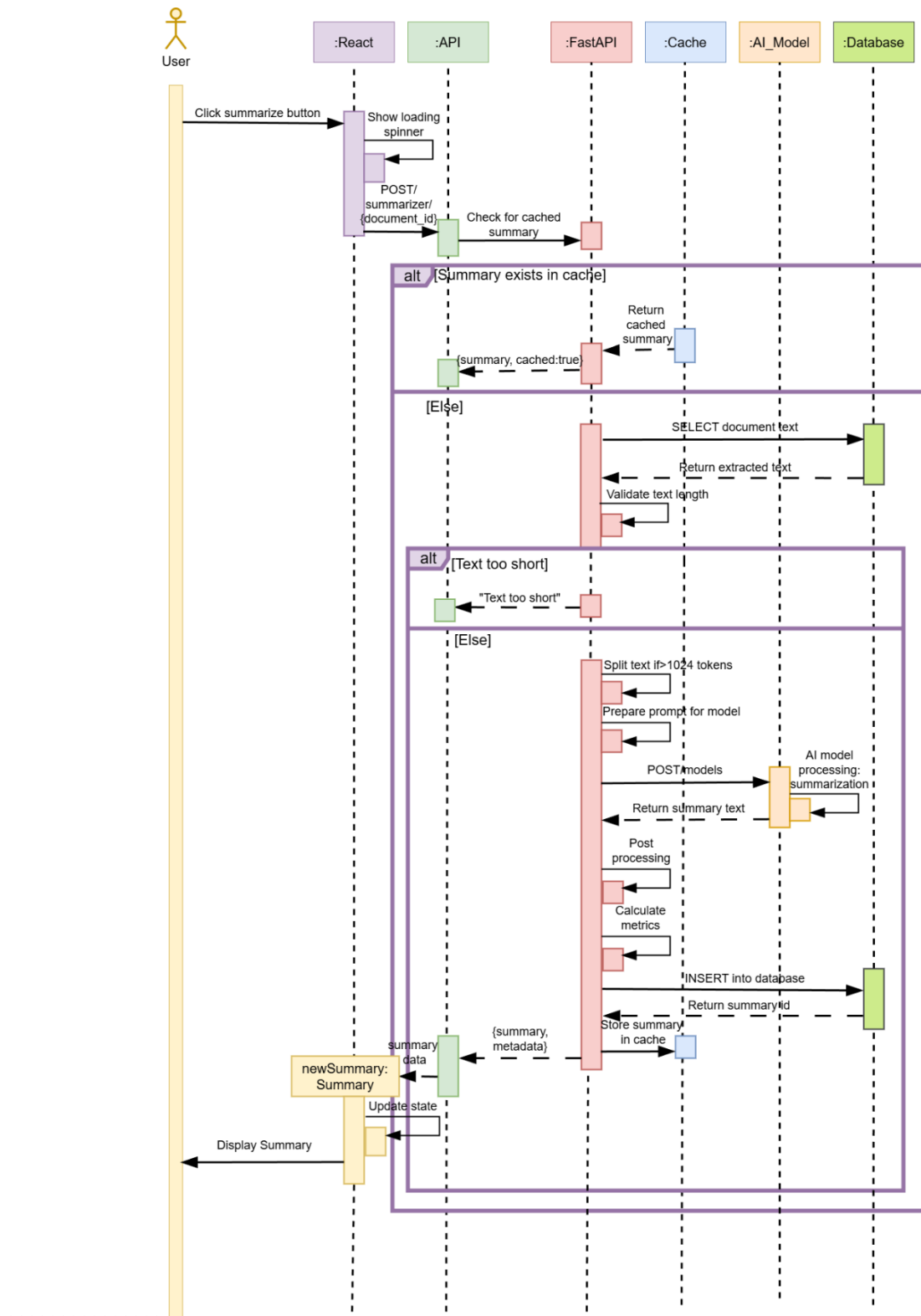
Registration and Login Sequence Diagram



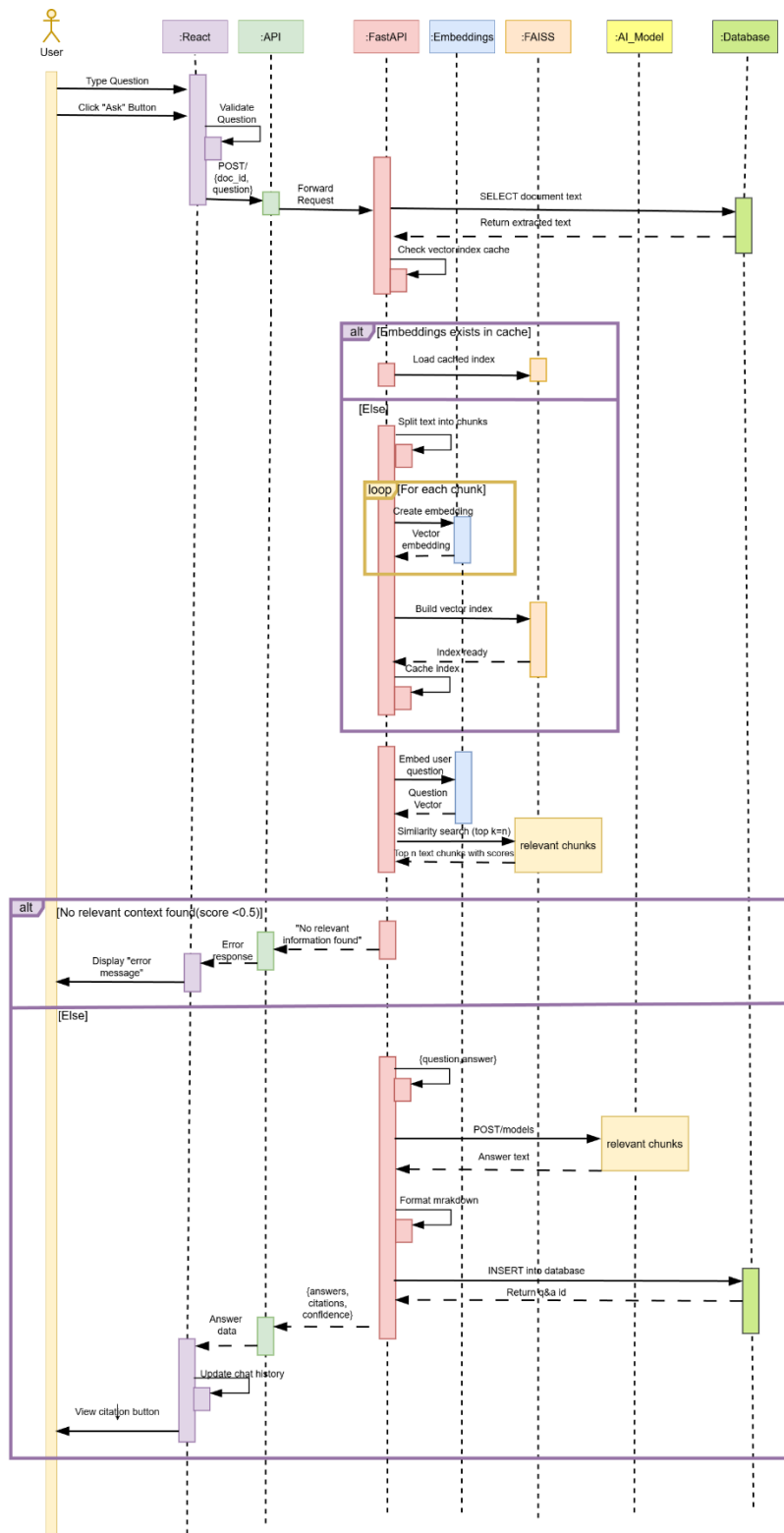
PDF upload and processing Seq Diagram



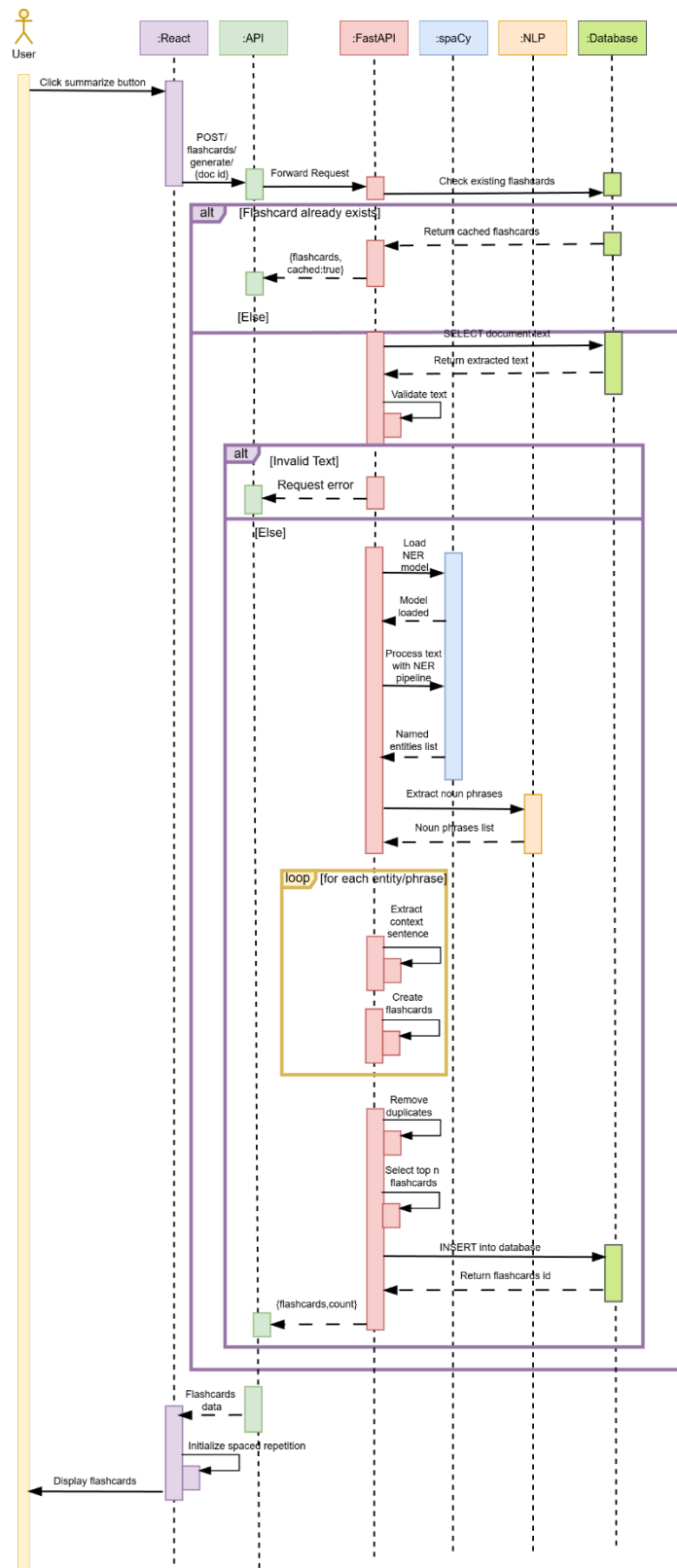
Document Summarization Seq Diagram



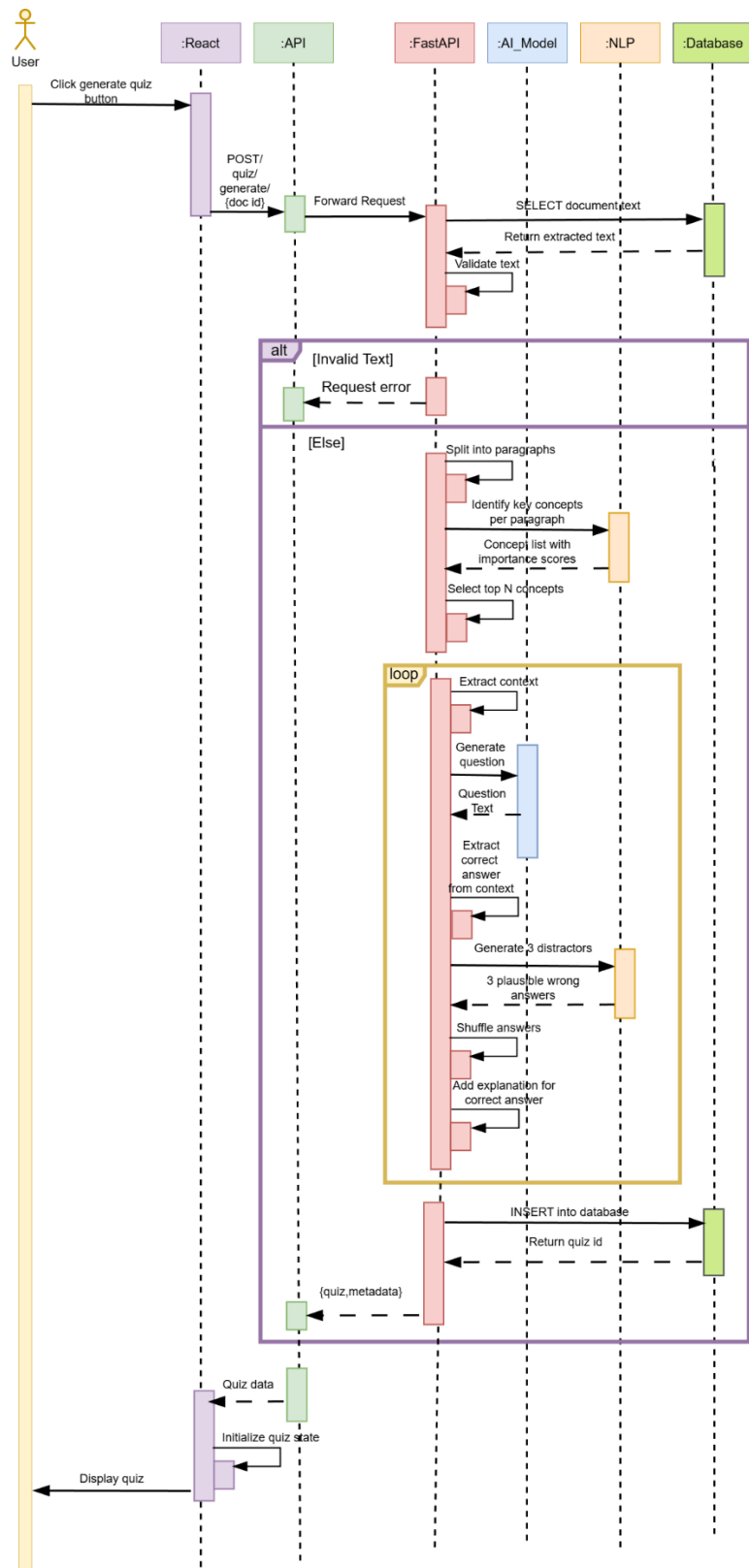
Q&A Chatbot Diagram



Flashcards creation Sequence Diagram

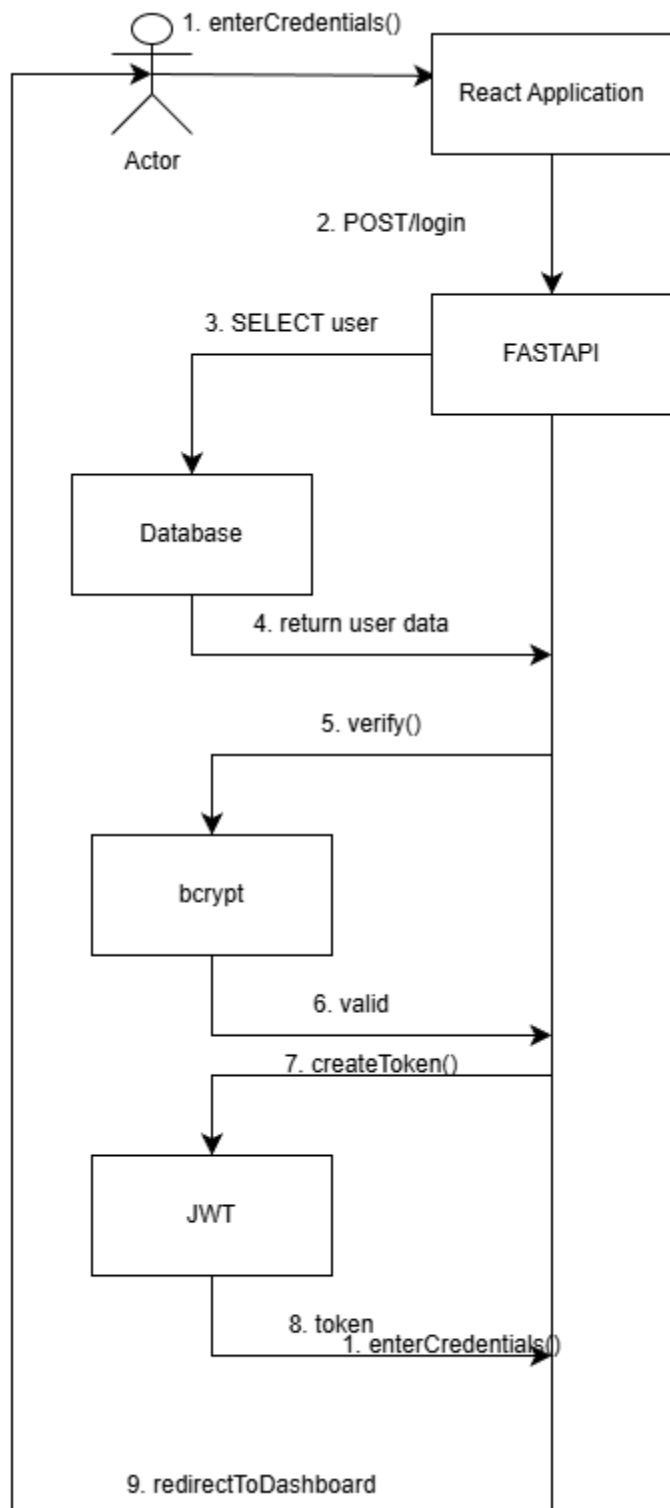


Quiz creation Sequence Diagram

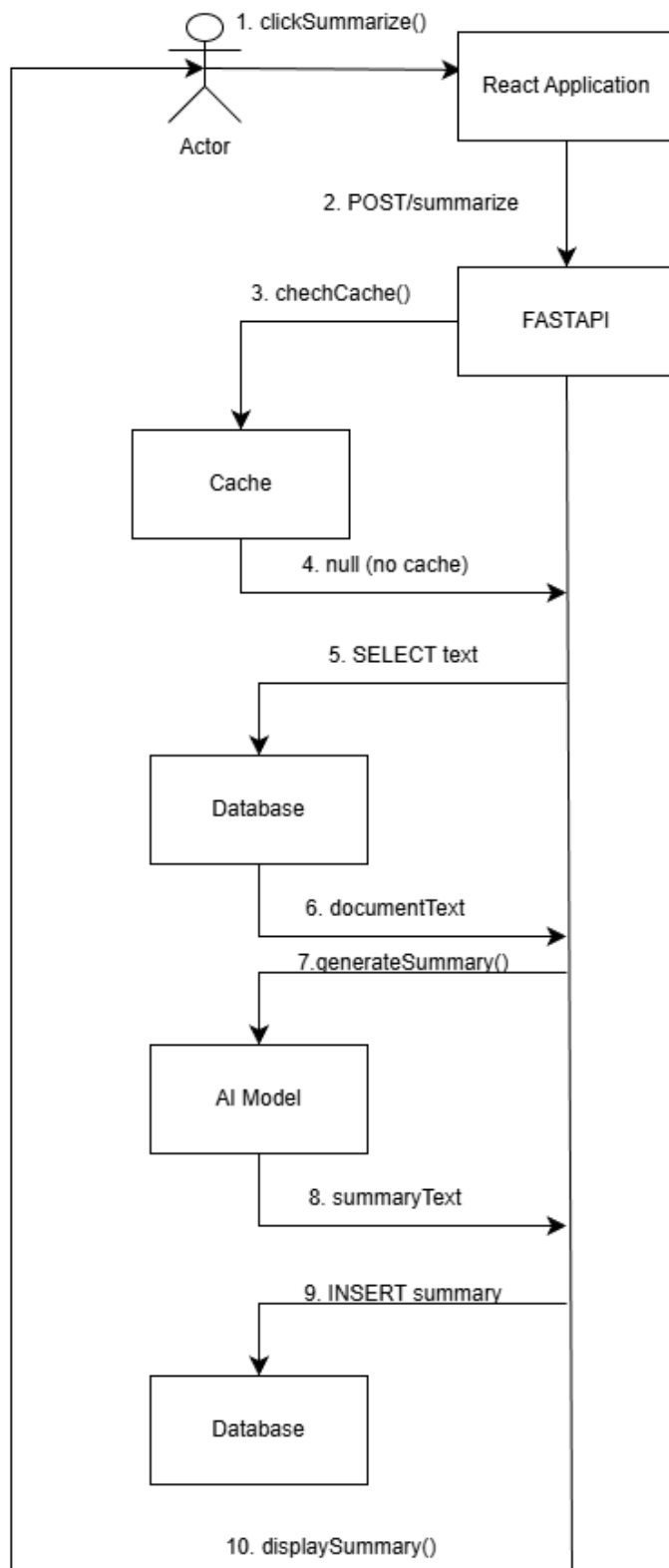


3.3.5 Collaboration Diagram

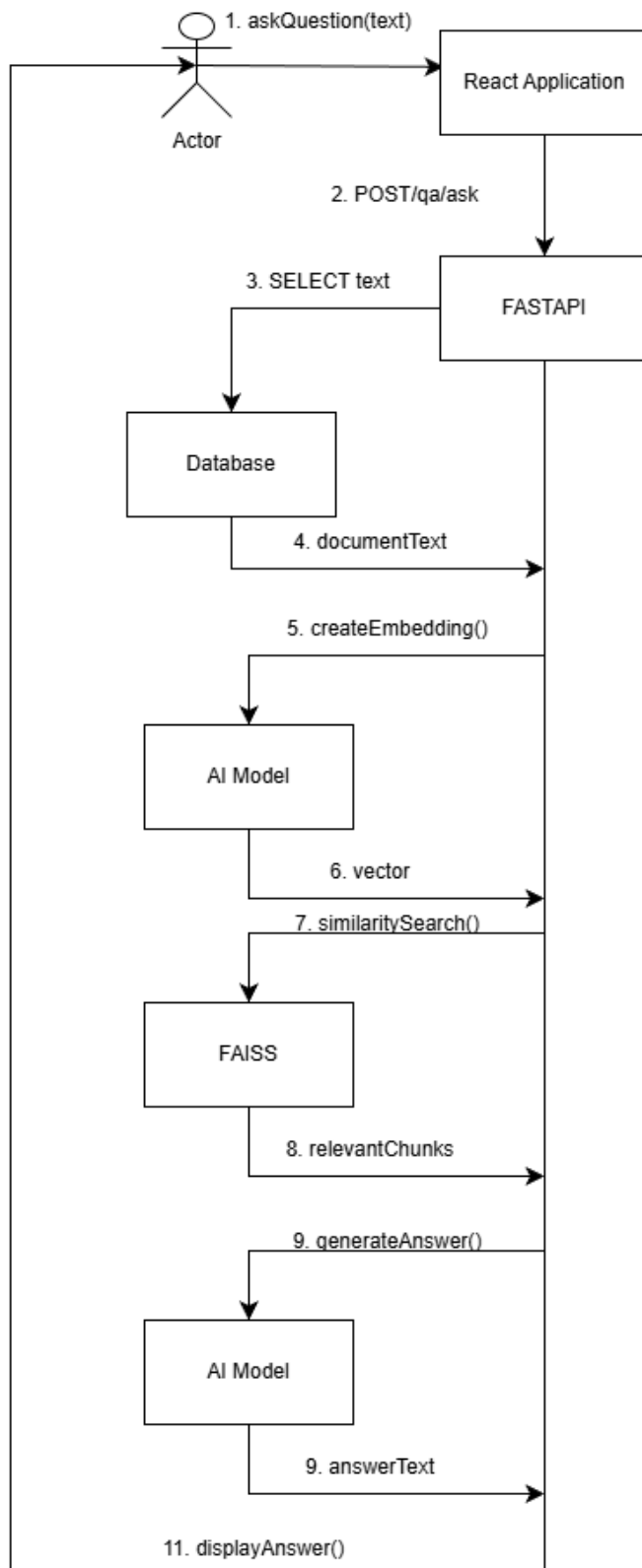
Registration and Login Collaboration Diagram



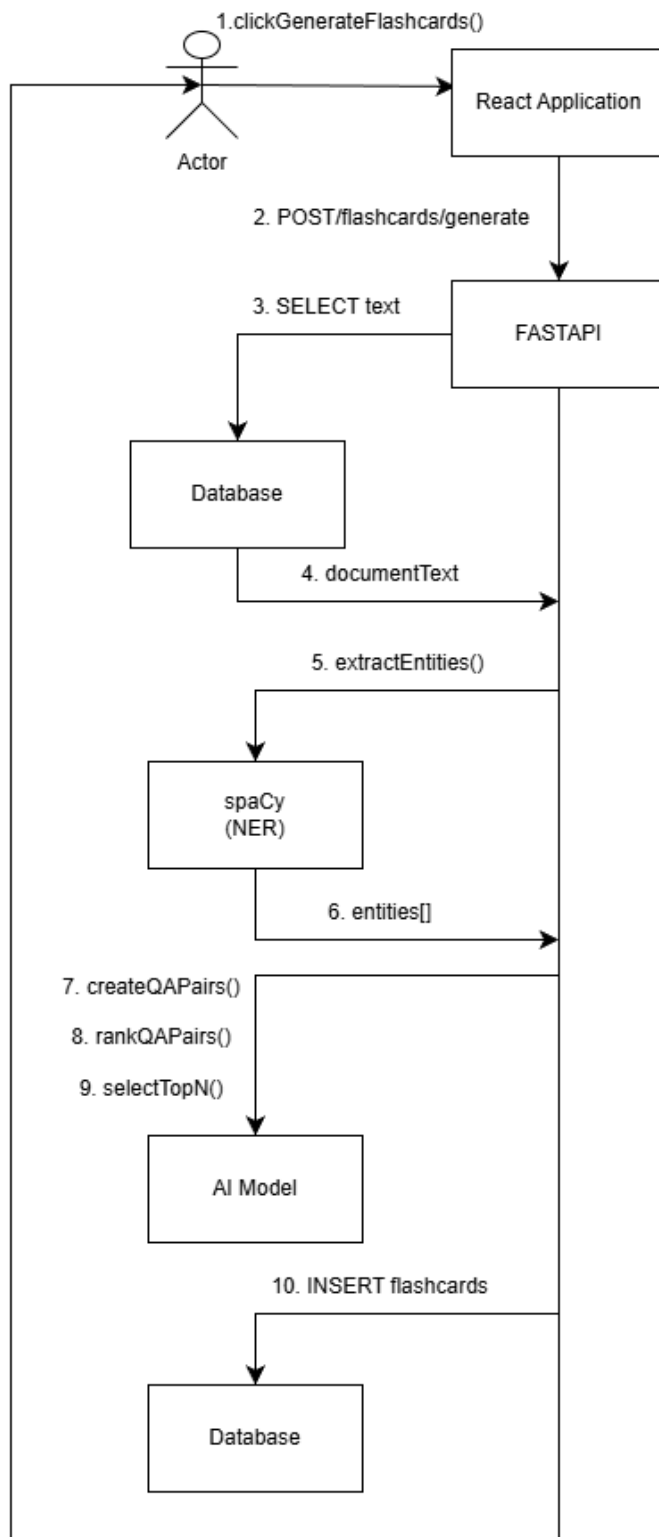
Summarizer Collaboration Diagram



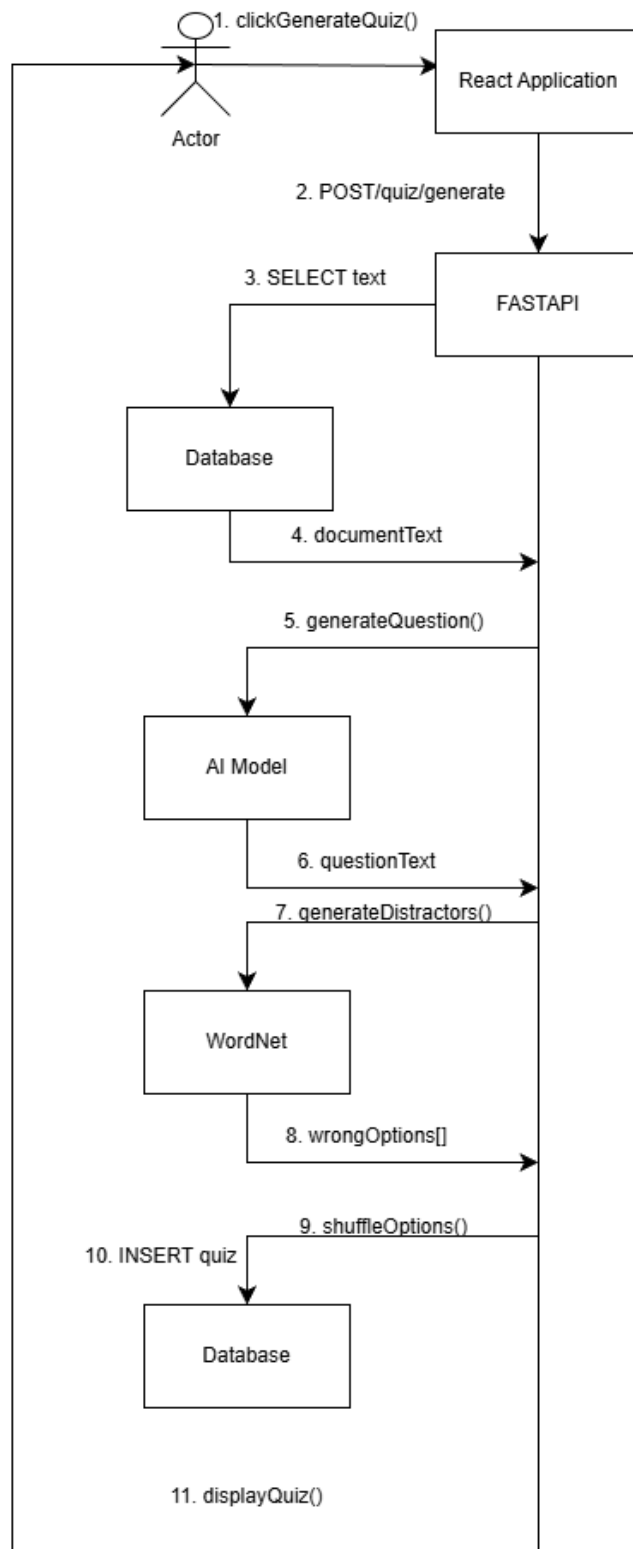
Q&A Chatbot Collaboration Diagram



Generate Flashcards Collaboration Diagram



Generate Quiz Collaboration Diagram



3.3.5 Entity Relationship Diagram

3.3.6 Class Diagram

3.3.7 Wireframe Designs

- Login/Registration Page
- Dashboard with Document Library
- Document Upload Interface
- Summary View Page
- Interactive Q&A Interface
- Flashcard Review System
- Practice Quiz Interface
- Progress Analytics Dashboard

3.4 Project Management

The project is being managed using Trello for task tracking and sprint planning.

4. PROGRESS ANALYSIS

4.1 Overview of Current Status

As of this interim report, the project has completed the planning and initial development phases. The following components have been implemented:

Completed Components:

- Project proposal and requirement analysis
- System architecture design
- Development environment setup
- Initial model integration research and testing
- Database schema design
- Login Page
- Registration Page
- User authentication system

In Progress:

Planned for Next Phase:

4.2 Comparison with Project Plan

The project is currently aligned with the planned timeline outlined in the proposal. The initial planning and setup phases were completed on schedule. Development of core features is proceeding as planned, with the summarization feature on track for completion by the end of the current sprint.

Some minor adjustments have been made to the development sequence based on technical dependencies. The authentication system was prioritized earlier than originally planned to enable early user testing. This adjustment has not impacted the overall timeline and will benefit the testing phase.

4.3 Challenges Encountered

4.4 Successes and Achievements

- Established efficient development workflow using Agile Scrum methodology
- Created comprehensive project documentation and technical specifications
- Received positive feedback from supervisors on project scope and approach
- Completed initial user research providing valuable insights for feature prioritization

4.5 Analysis of Development Activities

Development activities have been systematically organized following the Agile Scrum framework. Each sprint focuses on specific features with clear deliverables. Regular supervisor meetings provide valuable feedback and ensure alignment with academic requirements.

Frontend development is progressing smoothly with React.js proving to be an excellent choice for creating a responsive, modern interface. Component-based architecture facilitates rapid iteration and testing.

4.6 Progress Table

Phase	Planned Completion	Actual Completion	Status
Proposal & Planning	Week 1-2	Week 1-2	✓ Complete
Requirements Analysis	Week 3	Week 3	✓ Complete
System Design	Week 4-5	Week 4-5	✓ Complete
Environment Setup	Week 6	Week 6	✓ Complete
User Authentication System	Week 6	Week 6	✓ Complete
Summarization Feature	Week 7-8	Week 7-8 (ongoing)	🔄 In Progress
Q&A System	Week 9-10	Not started	○ Planned
Flashcard Generation	Week 11-12	Not started	○ Planned
MCQ Generation	Week 13-14	Not started	○ Planned
Frontend Development	Week 6-16	Week 6-8 (ongoing)	🔄 In Progress

Phase	Planned Completion	Actual Completion	Status
Testing & Validation	Week 17-18	Not started	○ Planned
Final Documentation	Week 19-20	Not started	○ Planned

5. Future Work